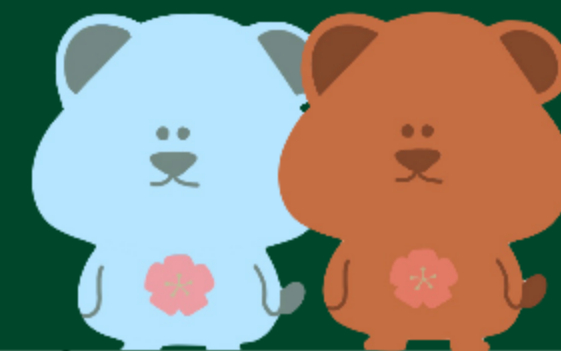




6. 합성곱 신경망 II

1팀 노현선 엄지민 이민경

6.1 이미지 분류를 위한 신경망

#01 LeNet-5

#02 AlexNet

#03 VGGNet

#04 GoogLeNet



1. LeNet-5



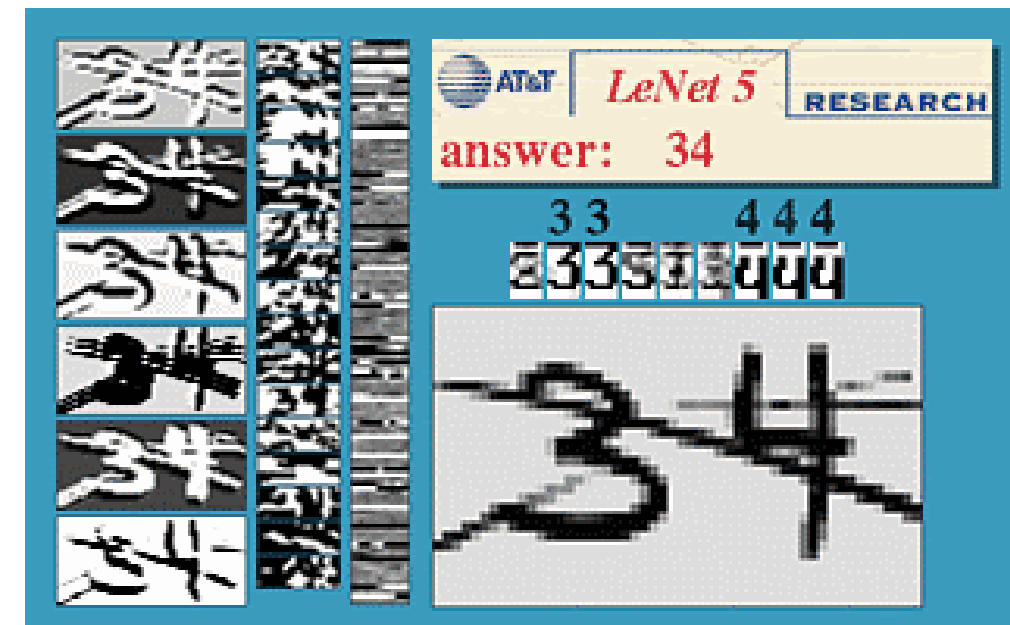
#6.1.1 LeNet-5

📌 이미지를 사용한 분류

- 특정 대상이 영상 내에 존재하는지 여부를 판단하는 것
- 이미지 분류에서 주로 사용되는 합성곱 신경망: **LeNet-5**, AlexNet, VGGNet, GoogLeNet...

📌 LeNet-5

- 1995년 발표되어, 현재 CNN의 초석이 된 딥러닝 구조
- 수표에 적힌 손글씨 숫자를 인식하던 모델 by 얀 르쿤
- 합성곱과 다운 샘플링(풀링)을 반복적으로 거치면서 마지막 완전연결층에서 분류 수행

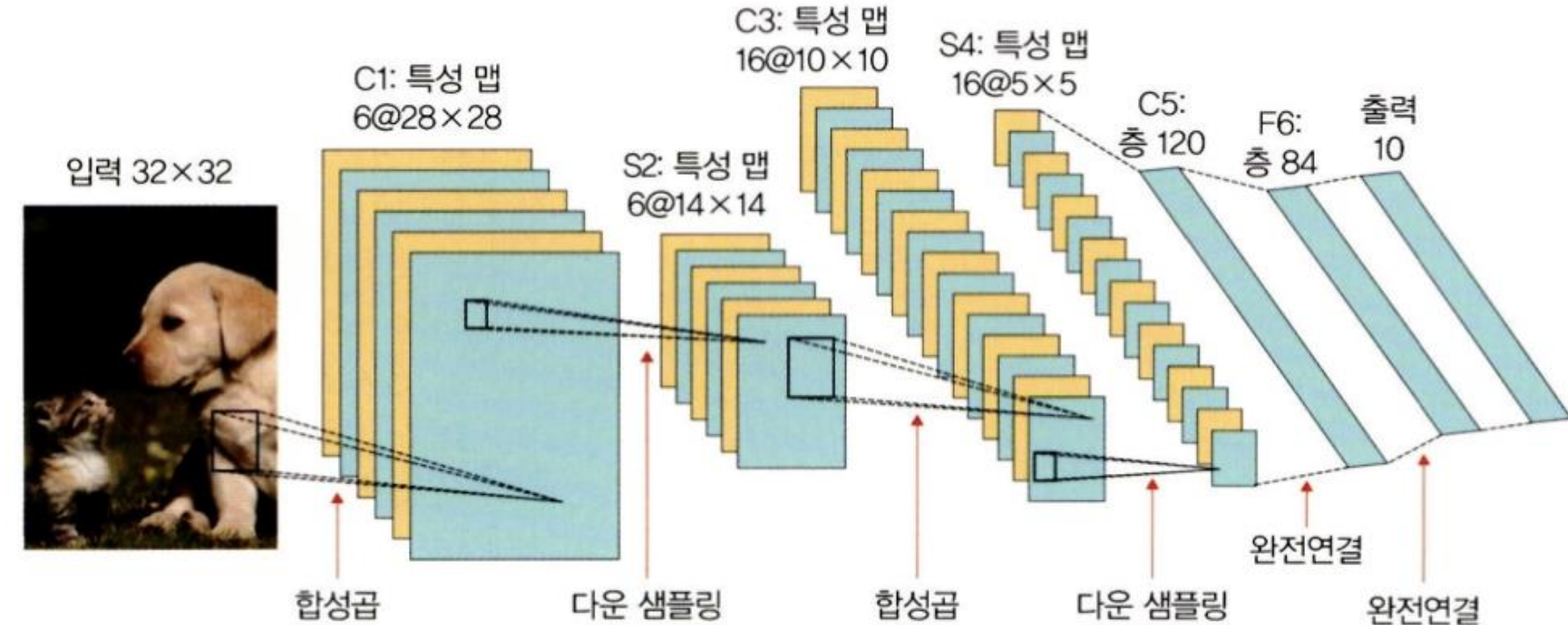


6.1.1 LeNet-5

🔑 주요 레이어별 역할

- ① Conv: 이미지에서 특징 추출
- ② Pooling: 크기 축소 + 중요한 정보 유지
- ③ 반복: 더 추상적인 특징 학습
- ④ FC: 최종 분류 수행

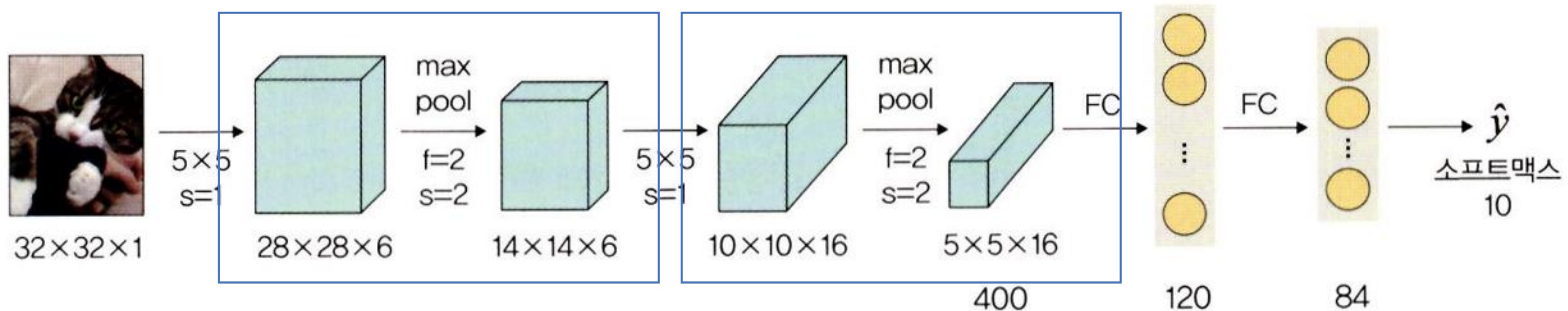
이미지를 점점 작게 줄이면서 중요한 특징만 남기고, 마지막에 분류하는 구조



C: 합성곱층, S: 풀링층, F: 완전연결층

6.1.1 LeNet-5

📌 LeNet-5 구현하기



크기가 32×32 인 이미지에 합성곱층(c)과 최대 풀링층(s)이 쌍으로 두 번 적용된 후, 완전연결층(f)을 거쳐 이미지가 분류되는 신경망

6.1.1 LeNet-5

🔑 라이브러리 호출

```
import torch
import torchvision
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms # 이미지 전처리 기능 제공
from torch.autograd import Variable
from torch import optim
import torch.nn as nn
import torch.nn.functional as F
import os
import cv2
from PIL import Image
from tqdm import tqdm_notebook as tqdm # 진행 상황을 가시적으로 표현. 모델의 학습 경과 확인할 때 사용.
import random
from matplotlib import pyplot as plt

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

transforms: 이미지 전처리 기능을 제공하는 라이브러리 (torchvision 제공)
tqdm: 진행 상황 시각화 라이브러리

6.1.1 LeNet-5

📌 이미지 데이터셋 전처리: ImageTransform 클래스

```
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = { Compose: 이미지를 변형할 수 있는 방식의 묶음
                                'train': transforms.Compose([
                                    transforms.RandomResizedCrop(resize, scale=(0.5, 1.0)),
                                    transforms.RandomHorizontalFlip(),
                                    transforms.ToTensor(),
                                    transforms.Normalize(mean, std)
                                ]),
                                'val': transforms.Compose([
                                    transforms.Resize(256),
                                    transforms.CenterCrop(resize),
                                    transforms.ToTensor(),
                                    transforms.Normalize(mean, std)
                                ])
                            }

    def __call__(self, img, phase):
        return self.data_transform[phase](img)
```

전처리 방식이 다름

1. 훈련: 데이터 늘리기
2. 검증: 성능 평가 위해 변형 x

RandomResizedCrop: 입력 이미지를 주어진 크기로 조정함. scale에 적힌 수의 크기로 이미지의 면적을 무작위로 자름.

RandomHorizontalFlip: 주어진 확률로 이미지를 수평 반전. (기본값 0.5)

ToTensor: 효율적 연산을 위해 이미지를 텐서 배열로 변환. 픽셀값 범위가 [0.0, 1.0]이 됨. (기본 토치비전: Python Image Library, [0, 255])

***Normalize:** ImageNet 데이터의 각 채널별 평균과 표준편차에 맞게 정규화

클래스 호출용 메서드: 클래스 객체를 호출되면 이 함수의 리턴 값이 반환됨.
phase='train' 또는 phase='val'로 이후 지정.

6.1.1 LeNet-5

📌 이미지 데이터셋 불러온 후 훈련, 검증, 테스트로 분리

```
cat_directory = 'dogs-vs-cats/Cat/'
dog_directory = 'dogs-vs-cats/Dog/'

# 1. 데이터를 정렬된 리스트로 만들어 반환하고, 지정한 파일 디렉터리 내 모든 파일의 리스트 반환
cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in os.listdir(cat_directory)])
dog_images_filepaths = sorted([os.path.join(dog_directory, f) for f in os.listdir(dog_directory)])

# 개와 고양이 이미지들을 합쳐서 image_filepaths에 저장
images_filepaths = [*cat_images_filepaths, *dog_images_filepaths]

# 2. 위의 image_filepaths에서 데이터가 존재하지 않을 때까지 반복하여 이미지 데이터를 불러옴
correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not None]

# 3. 난수 결과 고정 및 데이터 셔플
random.seed(42)
random.shuffle(correct_images_filepaths)

train_images_filepaths = correct_images_filepaths[:400] # 훈련용 이미지 400개
val_images_filepaths = correct_images_filepaths[400:-10] # 검증용 이미지 92개
test_images_filepaths = correct_images_filepaths[-10:] # 테스트용 이미지 10개

print(len(train_images_filepaths), len(val_images_filepaths), len(test_images_filepaths))
```

400 92 10

EWCHA
EUROPEAN

🔴 난수 결과 고정 및 데이터 셔플

3. 난수 결과 고정 및 데이터 셔플

```
random.seed(42)
random.shuffle(correct_images_filepaths)
```

seed 함수에 특정 값을 부여하면 상태가 저장되어 동일한 난수를 생성함.

→ 데이터를 섞되, 매번 같은 순서로 섞는 것.

그렇지 않을 경우 매 실행마다 훈련, 검증 테스트 데이터 바뀜.

성능 저하 및 실험 재현 불가.

*데이터 셔플 전: 앞에는 고양이, 뒤에는 개가 몰려 있음. (데이터 편향)

임의 추가: 라벨 확인 (편향성 판단)

```
labels = ['cat' if 'Cat' in path else 'dog' for path in images_filepaths]
```

```
print(labels[:30])
```

```
print(labels[-30:])
```

[illegible]

*데이터 셔플 후: 개와 고양이 라벨이 골고루 섞여 있음.

임의 추가: 셔플 후 라벨 출력

```
labels = ['cat' if 'Cat' in path else 'dog' for path in correct_images_filepaths]
```

```
print(labels[:50])
```

```
['cat', 'cat', 'dog', 'cat', 'cat', 'cat', 'cat', 'dog', 'cat', 'dog', 'cat', 'dog', 'dog', 'cat', 'dog',
```

6.1.1 LeNet-5

📌 DogvsCatDataset

```
class DogvsCatDataset(Dataset):
```

```
# 이미지 경로 리스트, 전처리 함수, 훈/검/테 분리
```

```
def __init__(self, file_list, transform=None, phase='train'):
```

```
    self.file_list = file_list #
```

```
    self.transform = transform
```

```
    self.phase = phase
```

파라미터: 이미지 경로 리스트, 전처리 함수, 데이터셋 분리(훈/검/테)

```
# images_filepaths 데이터셋의 전체 길이 반환
```

```
def __len__(self):
```

```
    return len(self.file_list)
```

```
# 지정된 경로의 이미지 읽어오기: 결과는 텐서의 형태
```

```
def __getitem__(self, idx):
```

```
    img_path = self.file_list[idx]
```

```
    img = Image.open(img_path)
```

```
    img_transformed = self.transform(img, self.phase) # 이미지에 'train' 전처리 적용
```

```
# 이미지 데이터에 대한 레이블 값(dog, cat) 가져오기
```

```
# dog은 1, cat은 0
```

```
label = img_path.split('/')[-1].split('.')[0] # 경로에서 슬래시와 온점 제거
```

```
if label == 'dog':
```

```
    label = 1
```

```
elif label == 'cat':
```

```
    label = 0
```

```
return img_transformed, label
```

__init__: 학습할 데이터의 경로 정의

__getitem__: 지정된 경로에서 이미지 읽어오기

dataloader: 데이터셋에서 데이터를 배치 크기만큼 메모리 불러옴

6.1.1 LeNet-5

📌 DogvsCatDataset 객체 생성

위에서 정의한 DogvsCatDataset 클래스에 대한 객체 생성(훈련, 검증)
전처리도 함께 적용

파라미터: 이미지 경로 리스트, 전처리 함수, 데이터셋 분리(훈/검/테)

```
train_dataset = DogvsCatDataset(train_images_filepaths, transform=ImageTransform(size, mean, std), phase='train')  
val_dataset = DogvsCatDataset(val_images_filepaths, transform=ImageTransform(size, mean, std), phase='val')
```

```
class ImageTransform():  
    def __init__(self, resize, mean, std):  
        self.data_transform = {  
            'train': transforms.Compose([  
                transforms.RandomResizedCrop(resize, scale=(0.5, 1.0)),  
                transforms.RandomHorizontalFlip(),  
                transforms.ToTensor(),  
                transforms.Normalize(mean, std)  
            ]),  
            'val': transforms.Compose([  
                transforms.Resize(256),  
                transforms.CenterCrop(resize),  
                transforms.ToTensor(),  
                transforms.Normalize(mean, std)  
            ])  
        }  
    }  
  
    def __call__(self, img, phase):  
        return self.data_transform[phase](img)
```

전처리 시 사용할 변수에 대한 값 정의
size = 224
mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)
batch_size = 32

Dataset → transform 호출 → phase 전달 → 전처리 선택

- train: 데이터 다양화 (RandomCrop, Flip)
- val: 데이터 고정 (평가 일관성 유지)

6.1.1 LeNet-5

📌 LeNet 모델 구축: 클래스 정의

```
class LeNet(nn.Module):  
    def __init__(self):  
        super(LeNet, self).__init__()  
        self.cnn1 = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=5, stride=1, padding=0) # 2D 합성곱층  
        self.relu1 = nn.ReLU() # 활성화 함수  
        self.maxpool1 = nn.MaxPool2d(kernel_size=2) # 최대 풀링  
  
        self.cnn2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=5, stride=1, padding=0)  
        self.relu2 = nn.ReLU()  
        self.maxpool2 = nn.MaxPool2d(kernel_size=2)  
        self.fc1 = nn.Linear(32*53*53, 512)  
        self.relu5 = nn.ReLU()  
        self.fc2 = nn.Linear(512, 2)  
        self.output = nn.Softmax(dim=1)  
  
    def forward(self, x):  
        out = self.cnn1(x)  
        out = self.relu1(out)  
        out = self.maxpool1(out)  
        out = self.cnn2(out)  
        out = self.relu2(out)  
        out = self.maxpool2(out)  
        out = out.view(out.size(0), -1)  
        out = self.fc1(out)  
        out = self.relu5(out)  
        out = self.fc2(out)  
        out = self.output(out)  
        return out
```

*LeNet: 합성곱층과 풀링층을 반복적으로 거치며 완전연결층에서 최종 분류 수행

1차 합성곱 + ReLU + 최대 풀링
→ 2차 합성곱 + ReLU + 최대 풀링
→ 특성맵 펼치기(flatten)
→ 완전연결층
→ 최종 2개 클래스 분류

6.1.1 LeNet-5

📌 네트워크 구조 요약: torchsummary

```
# 모델 객체 생성
model = LeNet().to(device)
print(model)

LeNet(
  (cnn1): Conv2d(3, 16, kernel_size=(5, 5), stride=(1, 1))
  (relu1): ReLU()
  (maxpool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (cnn2): Conv2d(16, 32, kernel_size=(5, 5), stride=(1, 1))
  (relu2): ReLU()
  (maxpool2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=89888, out_features=512, bias=True)
  (relu5): ReLU()
  (fc2): Linear(in_features=512, out_features=2, bias=True)
  (output): Softmax(dim=1)
)
```

summary(모델 네트워크, 입력값=(채널, 너비, 높이))

- 네트워크 내 파라미터 수와 구조를 쉽게 표현
- 총 파라미터 수, 입력 크기, 네트워크의 총 크기 등...

```
# 위와 같은 출력 결과가 눈에 안 들어올 때 사용하는 모델 구조 출력 라이브러리
# 파라미터: (모델 네트워크, 입력 값 c w h)
from torchsummary import summary
summary(model, input_size=(3, 224, 224))
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 16, 220, 220]	1,216
ReLU-2	[-1, 16, 220, 220]	0
MaxPool2d-3	[-1, 16, 110, 110]	0
Conv2d-4	[-1, 32, 106, 106]	12,832
ReLU-5	[-1, 32, 106, 106]	0
MaxPool2d-6	[-1, 32, 53, 53]	0
Linear-7	[-1, 512]	46,023,168
Linear-8	[-1, 2]	1,026
Softmax-9	[-1, 2]	0

```
=====  
Total params: 46,038,242  
Trainable params: 46,038,242  
Non-trainable params: 0  
=====
```

```
Input size (MB): 0.57  
Forward/backward pass size (MB): 19.47  
Params size (MB): 175.62  
Estimated Total Size (MB): 195.67  
=====
```

6.1.1 LeNet-5

📌 Optimizer, 손실 함수

```
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
criterion = nn.CrossEntropyLoss()
```

옵티마이저: 모멘텀 SGD

- SGD(확률적 경사 하강법)에 모멘텀이 추가된 것.
- 매번 기울기를 구하지만, 가중치 수정 전에 이전 수정 방향(+, -)을 참고해 같은 방향으로 일정한 비율만 수정되도록 함.
- model.parameters: 경사 하강법을 통해 궁극적으로 업데이트하려는 파라미터. 가중치와 바이어스. 46,038,242개 파라미터.
- momentum: SGD를 적절한 방향으로 가속화하며 진동을 줄여 주는 파라미터.



Image 2: SGD without momentum



Image 3: SGD with momentum

SGD가 중구난방으로 최적점을 찾아가는 것에 대해서 그 진동을 줄이고, 최적점에 연관된 방향으로 더 가속함.

momentum=0.9: 이전 방향 90% 유지 + 현재 기울기 반영

6.1.1 LeNet-5

🔑 모델 학습 함수 정의

```
def train_model(model, dataloader_dict, criterion, optimizer, num_epoch):
    since = time.time()
    best_acc = 0.0

    for epoch in range(num_epoch):
        print('Epoch {}/{}'.format(epoch + 1, num_epoch))
        print('-'*20)

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train() # 학습 모드
            else:
                model.eval() # 평가 모드

            epoch_loss = 0.0
            epoch_corrects = 0

            # 배치 단위 데이터 반복
            for inputs, labels in tqdm(dataloader_dict[phase]):
                inputs = inputs.to(device)
                labels = labels.to(device)
                optimizer.zero_grad()
```

```
# train일 때만 gradient 계산
with torch.set_grad_enabled(phase == 'train'):
    outputs = model(inputs) # forward
    _, preds = torch.max(outputs, 1) # 예측 값 추출
    loss = criterion(outputs, labels) # loss 계산

    if phase == 'train': # train일 때만 역전파 및 가중치 업데이트
        loss.backward()
        optimizer.step()
```

```
# 손실 및 맞은 개수 누적
epoch_loss += loss.item() * inputs.size(0)
epoch_corrects += torch.sum(preds == labels.data)
```

```
# epoch 기준 평균 손실 및 정확도 계산
epoch_loss = epoch_loss / len(dataloader_dict[phase].dataset)
epoch_acc = epoch_corrects.double() / len(dataloader_dict[phase].dataset)
print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss, epoch_acc))
```

```
if phase == 'val' and epoch_acc > best_acc:
    best_acc = epoch_acc
    best_model_wts = model.state_dict()
```

- ① phase를 **train / val** 단계로 나누어 학습과 평가 수행
- ② forward → loss 계산 → (**train일 때만**) backward + update
- ③ 예측 값과 정답을 비교하여 정확도 계산층에서 최종 분류 수행

6.1.1 LeNet-5

📌 모델 훈련 및 성능 평가

```
Epoch 10/10
-----
100% ██████████ 13/13 [00:02<00:00, 5.54it/s]
train Loss: 0.6663 Acc: 0.5950
100% ██████████ 3/3 [00:00<00:00, 7.00it/s]
val Loss: 0.6925 Acc: 0.5109
Training complete in 0m 28s
Best val Acc: 0.510870
```

모델 테스트를 위한 함수 정의: 테스트 데이터셋 사용

```
import pandas as pd
```

```
id_list = []
pred_list = []
_id=0
```

훈련 데이터셋 사용한 학습과의 가장 큰 차이점

역전파 중 텐서들에 대한 변화도 계산할 필요 없음을 나타냄

```
with torch.no_grad():
    for test_path in tqdm(test_images_filepaths):
        img = Image.open(test_path)
        _id = test_path.split('/')[-1].split('.')[1]
        transform = ImageTransform(size, mean, std)
```

검증 데이터셋을 이용해 학습한 결과, 최고 정확도가 51%로 낮은 편. 테스트 데이터셋을 모델에 적용해 정확도 재측정.

테스트 데이터셋에 전처리 적용

```
img = transform(img, phase='val')
img = img.unsqueeze(0)
img = img.to(device)
```

```
model.eval()
```

```
outputs = model(img)
preds = F.softmax(outputs, dim=1)[:, 1].tolist()
id_list.append(_id)
pred_list.append(preds[0])
```

테스트 데이터셋의 예측 결과인 id와 label을 DF에 저장

```
res = pd.DataFrame({
    'id': id_list,
    'label': pred_list
})
```

```
res.head(10)
```

	id	label
0	109	0.493043
1	145	0.448217
2	15	0.646438
3	162	0.554052
4	167	0.569048
5	200	0.443007
6	210	0.625228
7	211	0.600985
8	213	0.476566
9	224	0.643829

6.1.1 LeNet-5

🔑 예측 결과 시각화

```
# 개와 고양이에 대한 클래스 정의
class_ = classes = {0:'cat', 1:'dog'}

# 테스트 데이터셋 이미지를 출력하기 위한 함수 정의
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12, 6))

    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        # res df의 id라는 컬럼에서 임의로 데이터 가져오기
        a = random.choice(res['id'].values)
        label = res.loc[res['id'] == a, 'label'].values[0]

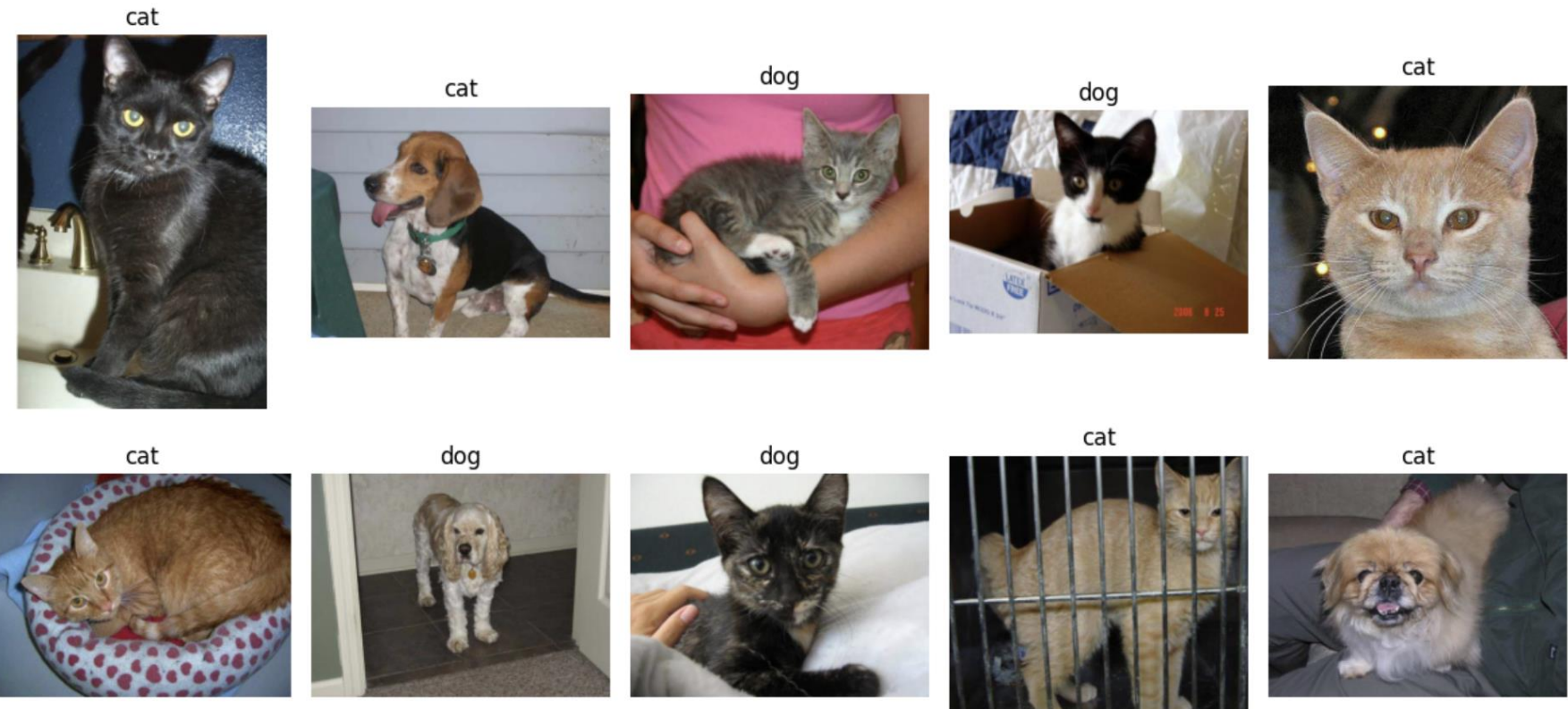
        if label > 0.5:
            label = 1 # dog
        else:
            label = 0 # cat

        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(class_[label])
        ax.ravel()[i].set_axis_off()

plt.tight_layout()
plt.show()
```

```
# 테스트 데이터셋의 예측 결과인 id와 label을 DF에 저장
res = pd.DataFrame({
    'id': id_list,
    'label': pred_list
})
```

display_image_grid(test_images_filepaths)



6.1.2 AlexNet



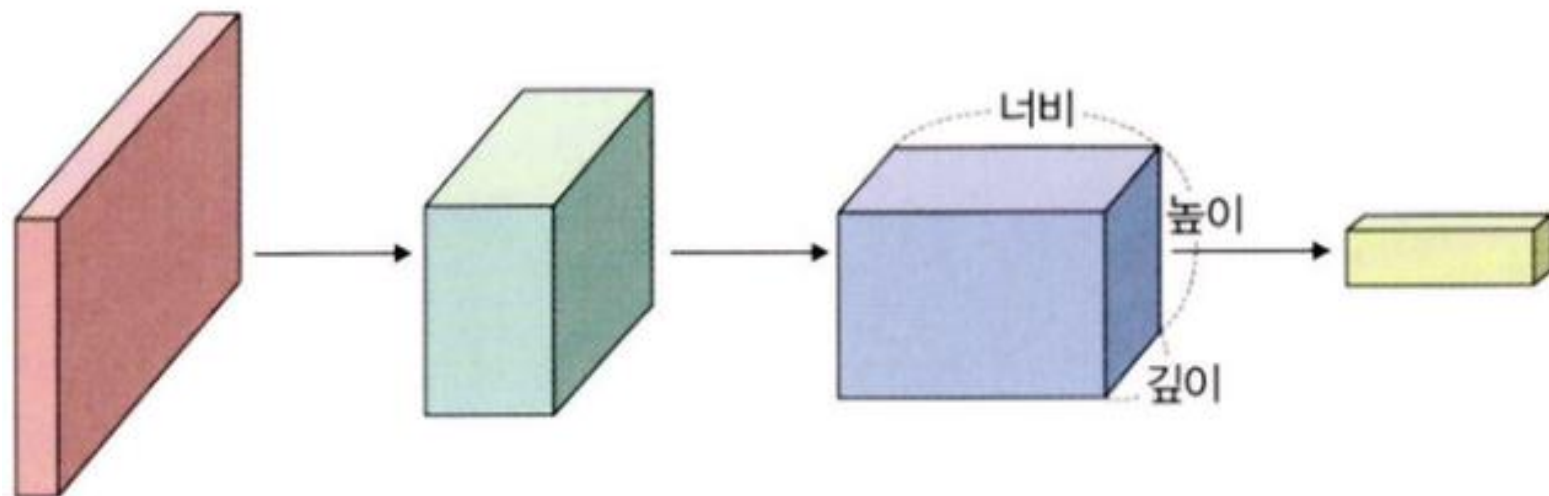
6.1.2 AlexNet

📌 Alexnet 이란?

- AlexNet은 ImageNet 영상 데이터베이스를 기반으로 한 화상 인식 대회인 ILSVRC 2012에서 우승한 CNN 구조

- CNN은 기본적으로 3차원 데이터를 다루는 구조
- 이미지는 R/G/B 성분 세 개를 갖기 때문에 시작이 3이지만, 합성곱을 거치면서 특성 맵이 만들어지고 이에 따라 중간 영상의 깊이가 달라짐

▼ 그림 6-8 CNN 구조

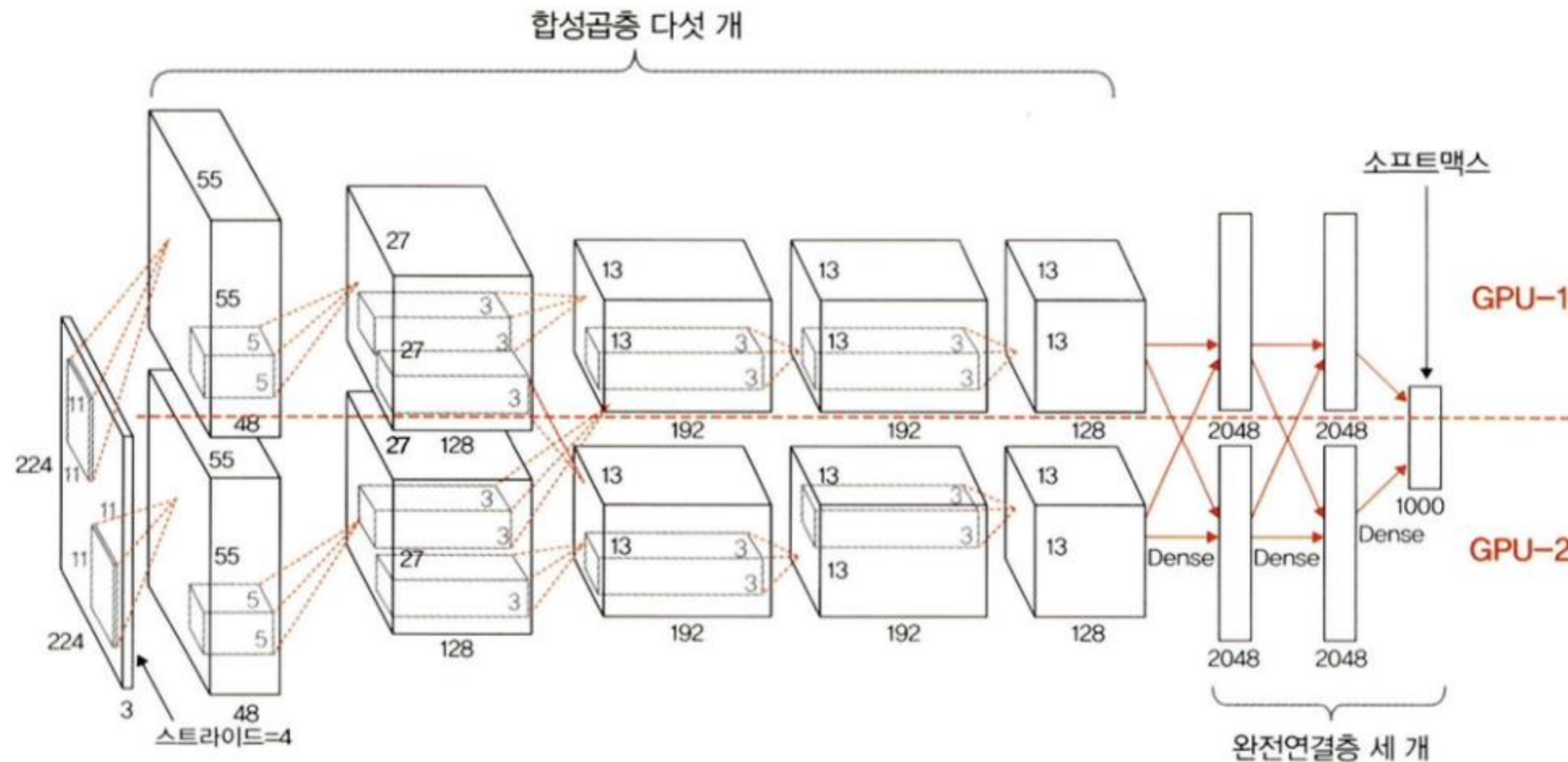


6.1.2 AlexNet

📌 Alexnet 구조

-합성곱층 5개 완전 연결층 3개로 구성

▼ 그림 6-9 AlexNet 구조



계층 유형	출력 크기	커널 크기	합성곱 수
Conv2d (1)	55×55×96	11×11, stride=4	4
Conv2d (2)	27×27×256	5×5	2
Conv2d (3)	13×13×384	3×3	1
Conv2d (4)	13×13×384	3×3	1
Conv2d (5)	13×13×256	3×3	1
FC (1)	4096	-	-
FC (2)	4096	-	-
FC (출력)	num_classes	-	-

6.1.2 AlexNet

Alexnet 구조

첫 번째 합성곱층의 특징

- 커널 크기: **11×11** , 스트라이드 4로 적용하여 **96개의 특성 맵** 생성
- **GPU-1**: 주로 컬러와 상관없는 정보(형태, 엣지 등)를 추출하기 위한 커널 학습
- **GPU-2**: 주로 컬러와 관련된 정보를 추출하기 위한 커널 학습

6.1.2 AlexNet

📌 데이터 샘플 확인

```
cat_filepaths = sorted(glob.glob(os.path.join(train_cats_dir, '*.jpg')))
dog_filepaths = sorted(glob.glob(os.path.join(train_dogs_dir, '*.jpg')))

print(f'Cat 이미지: {len(cat_filepaths):,}장   예: {os.path.basename(cat_filepaths[0])}')
print(f'Dog 이미지: {len(dog_filepaths):,}장   예: {os.path.basename(dog_filepaths[0])}')

all_filepaths = [*cat_filepaths, *dog_filepaths]
correct_filepaths = [p for p in all_filepaths if cv2.imread(p) is not None]
print(f'\n정상 파일: {len(correct_filepaths):,}장 / 전체: {len(all_filepaths):,}장')

random.seed(42)
random.shuffle(correct_filepaths)

train_images_filepaths = correct_filepaths[:400]
val_images_filepaths   = correct_filepaths[400:410]
test_images_filepaths  = correct_filepaths[-10:]

print(f'\n훈련: {len(train_images_filepaths)}장 / 검증: {len(val_images_filepaths)}장 / 테스트: {len(test_images_filepaths)}장')
```

Dogs vs Cats 샘플 이미지

Cat



Cat



Cat



Dog



Dog



Dog



6.1.2 AlexNet

📌 전처리: ImageTransform

```
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale=(0.5, 1.0)),
                transforms.RandomHorizontalFlip(),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]),
            'val': transforms.Compose([
                transforms.Resize(256),
                transforms.CenterCrop(resize),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ])
        }

    def __call__(self, img, phase):
        return self.data_transform[phase](img)
```

■ train 단계

RandomResizedCrop + RandomHorizontalFlip
→ 데이터 증강(augmentation)

■ val/test 단계

Resize(256) + CenterCrop(224)
→ 고정 크기 보장

■ Normalize

ImageNet 기준 mean/std 적용

6.1.2 AlexNet

📌 커스텀 데이터 셋 클래스

```
class DogsCatsDataset(Dataset):
    def __init__(self, file_list, transform=None, phase='train'):
        self.file_list = file_list
        self.transform = transform
        self.phase = phase

    def __len__(self):
        return len(self.file_list)

    def __getitem__(self, idx):
        img_path = self.file_list[idx]
        img = Image.open(img_path).convert('RGB')
        img_transformed = self.transform(img, self.phase)
        # cat.0.jpg → 'cat' → 0, dog.0.jpg → 'dog' → 1
        label_str = os.path.basename(img_path).split('.')[0]
        label = 0 if label_str == 'cat' else 1
        return img_transformed, label
```

- torch.utils.data.Dataset 상속
- __len__: 전체 데이터셋 크기 반환
- __getitem__: 파일명 기준 레이블 자동 결정
cat.*.jpg → label = 0
dog.*.jpg → label = 1
- Image.open().convert("RGB")로 투명 채널 문제 방지

6.1.2 AlexNet

📌 DataLoader 설정

```
size      = 256
mean      = (0.485, 0.456, 0.406)
std       = (0.229, 0.224, 0.225)
batch_size = 32

train_dataset = DogsCatsDataset(train_images_filepaths, transform=ImageTransform(size, mean, std), phase='train')
val_dataset   = DogsCatsDataset(val_images_filepaths,   transform=ImageTransform(size, mean, std), phase='val')

train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=0)
val_dataloader   = DataLoader(val_dataset,   batch_size=batch_size, shuffle=False, num_workers=0)
dataloader_dict  = {'train': train_dataloader, 'val': val_dataloader}

inputs, labels = next(iter(train_dataloader))
print('입력 텐서 크기:', inputs.size())
print('레이블 샘플  :', labels[:8].tolist())
```

3] ✓ 0.0s

입력 텐서 크기: torch.Size([32, 3, 256, 256])
레이블 샘플 : [1, 0, 0, 1, 0, 0, 1, 0]

- size = 256(AlexNet 입력 크기)
- batch_size = 32
- mean = (0.485, 0.456, 0.406)
- std = (0.229, 0.224, 0.225)
(ImageNet 정규화 값)

→ 입력 텐서:

torch.Size([32, 3, 256, 256])

6.1.2 AlexNet

📌 AlexNet 네트워크 클래스 정의

```
class AlexNet(nn.Module):
    def __init__(self, num_classes=2):
        super(AlexNet, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2), nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(64, 192, kernel_size=5, stride=1, padding=2), nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(192, 384, kernel_size=3, stride=1, padding=1), nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, stride=1, padding=1), nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, stride=1, padding=1), nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
        )
        self.avgpool = nn.AdaptiveAvgPool2d((6, 6))
        self.classifier = nn.Sequential(
            nn.Dropout(),
            nn.Linear(256 * 6 * 6, 4096), nn.ReLU(inplace=True),
            nn.Dropout(),
            nn.Linear(4096, 4096), nn.ReLU(inplace=True),
            nn.Linear(4096, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x
```

```
model = AlexNet()
model.to(device)
print(model)
```

```
AlexNet(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(11, 11), stride=(4, 4), padding=(2, 2))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(64, 192, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(192, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): ...
```

■ features (합성곱 블록)
Conv×5 + MaxPool×3

■ avgpool
AdaptiveAvgPool2d((6,6))
→ 256×6×6 = 9,216

■ classifier (완전 연결 블록)
Dropout→FC(9216,4096)→ReLU
Dropout→FC(4096,4096)→ReLU
FC(4096, 2)

6.1.2 AlexNet

📌 AlexNet 네트워크 클래스 정의

```
class AlexNet(nn.Module):
    def __init__(self, num_classes=2):
        super(AlexNet, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2), nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(64, 192, kernel_size=5, stride=1, padding=2), nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(192, 384, kernel_size=3, stride=1, padding=1), nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, stride=1, padding=1), nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, stride=1, padding=1), nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
        )
        self.avgpool = nn.AdaptiveAvgPool2d((6, 6))
        self.classifier = nn.Sequential(
            nn.Dropout(),
            nn.Linear(256 * 6 * 6, 4096), nn.ReLU(inplace=True),
            nn.Dropout(),
            nn.Linear(4096, 4096), nn.ReLU(inplace=True),
            nn.Linear(4096, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x
```

```
model = AlexNet()
model.to(device)
print(model)
```

```
AlexNet(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(11, 11), stride=(4, 4), padding=(2, 2))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(64, 192, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(192, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): ...
```

AdaptiveAvgPool2d vs AvgPool2d

	AvgPool2d	AdaptiveAvgPool2d
지정 방식	커널·스트라이드·패딩 직접 설정	출력 크기만 지정
입력 크기 변동	수동 계산 필요	자동 계산
활용	일반 풀링	FC층 앞단 연결 시 유리

$$H_{out} = \left\lfloor \frac{H_{in} + 2 \times padding[0] - kernel_size[0]}{stride[0]} + 1 \right\rfloor$$

6.1.2 AlexNet

AlexNet 네트워크 구조 확인

```
!pip install torchsummary
from torchsummary import summary
summary(model, input_size=(3, 256, 256))
```

✓ 1.3s Python

Collecting torchsummary
 Downloading torchsummary-1.5.1-py3-none-any.whl.metadata (296 bytes)
 Downloading torchsummary-1.5.1-py3-none-any.whl (2.8 kB)
Installing collected packages: torchsummary
Successfully installed torchsummary-1.5.1

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 63, 63]	23,296
ReLU-2	[-1, 64, 63, 63]	0
MaxPool2d-3	[-1, 64, 31, 31]	0
Conv2d-4	[-1, 192, 31, 31]	307,392
ReLU-5	[-1, 192, 31, 31]	0
MaxPool2d-6	[-1, 192, 15, 15]	0
Conv2d-7	[-1, 384, 15, 15]	663,936
ReLU-8	[-1, 384, 15, 15]	0
Conv2d-9	[-1, 256, 15, 15]	884,992
ReLU-10	[-1, 256, 15, 15]	0
Conv2d-11	[-1, 256, 15, 15]	590,080
ReLU-12	[-1, 256, 15, 15]	0
MaxPool2d-13	[-1, 256, 7, 7]	0
AdaptiveAvgPool2d-14	[-1, 256, 6, 6]	0
Dropout-15	[-1, 9216]	0
Linear-16	[-1, 4096]	37,752,832
ReLU-17	[-1, 4096]	0

...

Forward/backward pass size (MB): 10.96
Params size (MB): 217.48
Estimated Total Size (MB): 229.19

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings...](#)

6.1.2 AlexNet

📌 훈련 함수: train_model

```
def train_model(model, dataloader_dict, criterion, optimizer, num_epoch):
    since = time.time()

    for epoch in range(num_epoch):
        print(f'Epoch {epoch+1}/{num_epoch}')
        print('-' * 25)

        for phase in ['train', 'val']:
            model.train() if phase == 'train' else model.eval()
            epoch_loss, epoch_corrects = 0.0, 0

            for inputs, labels in tqdm(dataloader_dict[phase], desc=phase):
                inputs, labels = inputs.to(device), labels.to(device)
                optimizer.zero_grad()

                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    loss = criterion(outputs, labels)
                    _, preds = torch.max(outputs, 1)
                    if phase == 'train':
                        loss.backward()
                        optimizer.step()

                epoch_loss += loss.item() * inputs.size(0)
                epoch_corrects += torch.sum(preds == labels.data)

            n = len(dataloader_dict[phase].dataset)
            print(f'{phase:5s} Loss: {epoch_loss/n:.4f} Acc: {epoch_corrects.double()/n:.4f}')

    elapsed = time.time() - since
    print(f'\n훈련 완료: {elapsed//60:.0f}분 {elapsed%60:.0f}초')
    return model
```

- train 단계
model.train()
loss.backward() + optimizer.step()

- val 단계
model.eval()
torch.set_grad_enabled(False)

- 매 에폭마다
Loss & Accuracy 출력

- 파라미터
SGD(lr=0.001, momentum=0.9)
CrossEntropyLoss

6.1.2 AlexNet

🔑 모델 훈련 결과

```
num_epoch = 10
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)

model = train_model(model, dataloader_dict, criterion, optimizer, num_epoch)
```

Epoch 1/10

train Loss: 0.6935 Acc: 0.4625

val Loss: 0.6937 Acc: 0.4000
Epoch 2/10

train Loss: 0.6931 Acc: 0.4425

val Loss: 0.6933 Acc: 0.5000
Epoch 3/10

train Loss: 0.6933 Acc: 0.5100

val Loss: 0.6927 Acc: 0.6000
Epoch 4/10

train Loss: 0.6932 Acc: 0.4775

val Loss: 0.6925 Acc: 0.6000
Epoch 5/10

train Loss: 0.6934 Acc: 0.4800

val Loss: 0.6923 Acc: 0.6000
Epoch 6/10

train Loss: 0.6930 Acc: 0.5300

- num_epoch = 10
- optimizer: SGD
lr=0.001, momentum=0.9
- criterion: CrossEntropyLoss

(A파트 LeNet과 중복되므로
간략히 설명)

6.1.2 AlexNet

🔑 테스트 예측

```
id_list, pred_list = [], []

model.eval()
with torch.no_grad():
    for test_path in tqdm(test_images_filepaths, desc='predicting'):
        img = Image.open(test_path).convert('RGB')
        # cat.42.jpg → '42'
        _id = os.path.basename(test_path).split('.')[1]

        img_t = ImageTransform(size, mean, std)(img, phase='val')
        img_t = img_t.unsqueeze(0).to(device)

        outputs = model(img_t)
        prob = F.softmax(outputs, dim=1)[:, 1].item()

        id_list.append(int(_id))
        pred_list.append(prob)

res = pd.DataFrame({'id': id_list, 'label': pred_list})
res.sort_values('id', inplace=True)
res.reset_index(drop=True, inplace=True)

print('예측 결과:')
print(res)
print(f'\ndog 예측(확률>0.5): {(res["label"]>0.5).sum()}장 / {len(res)}장')
```

```
예측 결과:
   id  label
0  195  0.494493
1   699  0.494716
2  2510  0.495269
3  2835  0.494793
4  3893  0.494650
5  7625  0.494922
6  7779  0.494262
7  8900  0.494975
8  9271  0.494474
9 10928  0.494567
```

```
dog 예측(확률>0.5): 0장 / 10장
```

■ torch.no_grad()
추론 시 기울기 계산 비활성화
→ 메모리 절약

■ F.softmax(outputs, dim=1)[:, 1]
dog(1) 클래스 확률 추출

6.1.2 AlexNet

📌 예측 결과 시각화

```
def display_image_grid(filepaths, probs, cols=5):
    rows = max(1, len(filepaths) // cols)
    fig, ax = plt.subplots(rows, cols, figsize=(14, 6))
    for i, (path, prob) in enumerate(zip(filepaths, probs)):
        img = cv2.cvtColor(cv2.imread(path), cv2.COLOR_BGR2RGB)
        true_label = os.path.basename(path).split('.')[0] # cat / dog
        pred_label = 'dog' if prob > 0.5 else 'cat'
        color = 'green' if true_label == pred_label else 'red'
        ax.ravel()[i].imshow(img)
        ax.ravel()[i].set_title(f'실제:{true_label}\n예측:{pred_label}({prob:.2f})', color=color, fontsize=8)
        ax.ravel()[i].axis('off')
    plt.suptitle('초록=정답 / 빨강=오답')
    plt.tight_layout()
    plt.show()

display_image_grid(test_images_filepaths, pred_list)
```



6.1.3 VGGNet



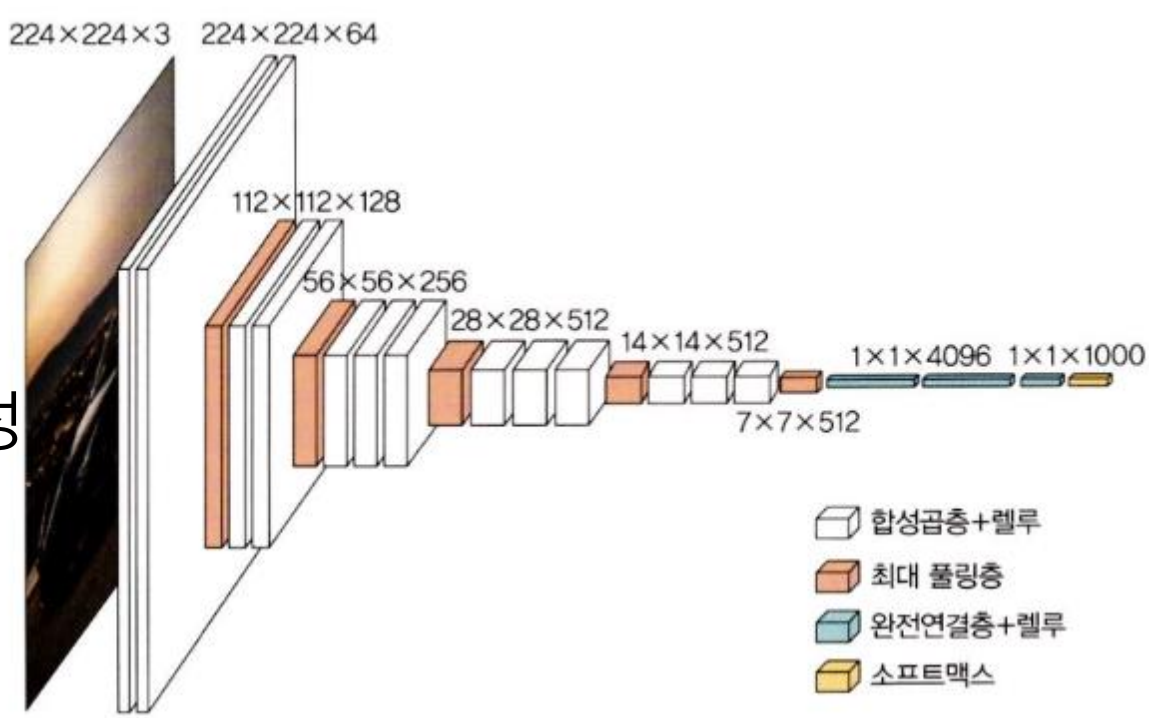
#6.1.3 VGGNet

목적: 합성곱층의 파라미터 수를 줄이고 훈련 시간 개선

-> 네트워크를 깊게 만드는것이 성능에 어떤 영향을 미치는지 확인
깊이의 영향만 최대한 확인하고자 합성곱층에서 사용하는 커널의 크기: 3x3로 고정

네트워크 계층의 총 갯수에 따라 여러 유형의 VGGNet
VGG11, VGG16, VGG19..

- VGG16:
- 총 1억 3300만 개의 파라미터
 - 모든 합성곱 커널의 크기: 3x3
 - 최대 풀링 커널의 크기: 2x2
 - 스트라이드: 2
 - > 64개의 224x224특성 맵 생성
 - 마지막 16번째 계층 제외하고 ReLU활성화 함수 적용



계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	224×224	—	—	—
합성곱층	64	224×224	3×3	1	렐루(ReLU)
합성곱층	64	224×224	3×3	1	렐루(ReLU)
최대 풀링층	64	112×112	2×2	2	—
합성곱층	128	112×112	3×3	1	렐루(ReLU)
합성곱층	128	112×112	3×3	1	렐루(ReLU)
최대 풀링층	128	56×56	2×2	2	—
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
최대 풀링층	256	28×28	2×2	2	—
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
최대 풀링층	512	14×14	2×2	2	—
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
최대 풀링층	512	7×7	2×2	2	—
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	1000	—	—	소프트맥스(softmax)

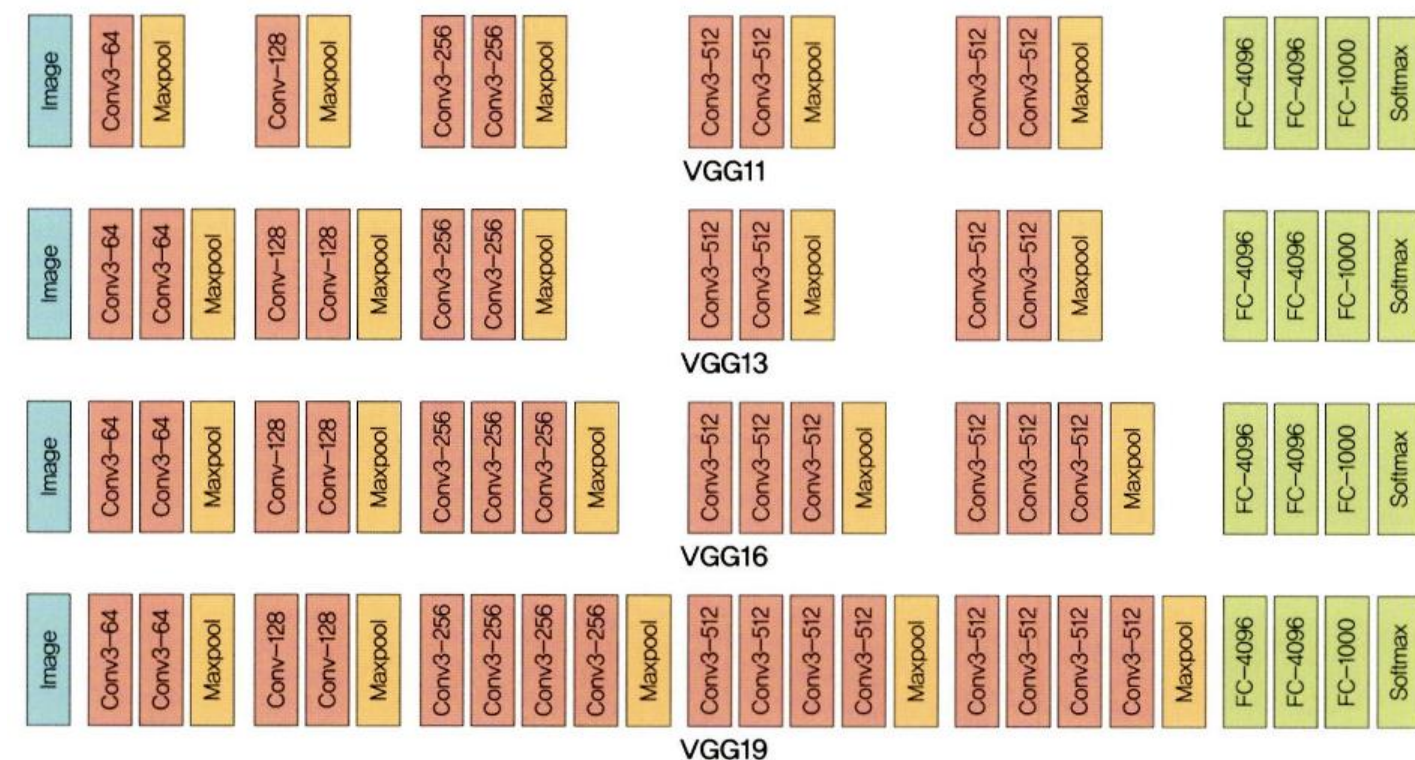
#6.1.3 VGGNet

VGG 11 구성 코드: vgg_config

```
class VGG(nn.Module):
    def __init__(self, features, output_dim):
        super().__init__()
        self.features = features
        self.avgpool = nn.AdaptiveAvgPool2d(7)
        self.classifier = nn.Sequential(
            nn.Linear(512*7*7, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(4096, output_dim)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.avgpool(x)
        h = x.view(x.shape[0], -1)
        x = self.classifier(h)
        return x, h
```

VGG모델에 대한 매개변수에서 받아 온 features 값을 self.features에 넣어 줌



```
vgg11_config = [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M']
vgg13_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M']
vgg16_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M', 512, 512, 'M']
vgg19_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512, 512, 512, 'M', 512, 512, 512, 512, 'M']
```

합성곱층 8개 + 풀링층 3개 = 총 11개 계층 : VGG11

VGG모델의 계층 정리:

- 숫자: Conv2d수행
- 출력채널 = 다음 계층의 입력 채널
- M : max pooling수행

#6.1.3 VGGNet

VGG 11 구성 코드: get_vgg_layers

```
def get_vgg_layers(config, batch_norm):
    layers = []
    in_channels = 3

    for c in config:
        assert c == 'M' or isinstance(c, int)
        if c == 'M':
            layers += [nn.MaxPool2d(kernel_size=2)]
        else:
            conv2d = nn.Conv2d(in_channels, c, kernel_size=3, padding=1)
            if batch_norm:
                layers += [conv2d, nn.BatchNorm2d(c), nn.ReLU(inplace=True)]
            else:
                layers += [conv2d, nn.ReLU(inplace=True)]
            in_channels = c

    return nn.Sequential(*layers)
```

vgg11_config값들을 가져옴

M이면 최대 풀링 적용

불러온 값이 숫자면 Conv2d적용

배치 정규화 적용될 경우는 배치 정규화+ReLU적용

배치 정규화 적용되지 않을 경우: ReLU만 적용

네트워크의 모든 계층 반환

#6.1.3 VGGNet

VGG 11 구성 코드:

배치 정규화 계층 추가해서
모델 계층 생성

```
vgg11_layers = get_vgg_layers(vgg11_config, batch_norm=True)
```

```
print(vgg11_layers)
```

```
Sequential(
  (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (2): ReLU(inplace=True)
  (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (4): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (5): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (6): ReLU(inplace=True)
  (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (8): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (9): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (10): ReLU(inplace=True)
  (11): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (12): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (13): ReLU(inplace=True)
  (14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (15): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (16): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (17): ReLU(inplace=True)
  (18): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (19): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (20): ReLU(inplace=True)
  (21): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (22): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (23): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (24): ReLU(inplace=True)
  (25): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (26): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (27): ReLU(inplace=True)
  (28): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
)
```

모델 전체에 대한 네트워크

OUTPUT_DIM = 2 2개의 클래스 사용: 개, 고양이
model = VGG(vgg11_layers, OUTPUT_DIM)
print(model)

```
VGG(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (4): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (5): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU(inplace=True)
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (8): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): ReLU(inplace=True)
    (11): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (12): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (13): ReLU(inplace=True)
    (14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (15): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (16): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (17): ReLU(inplace=True)
    (18): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (19): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (20): ReLU(inplace=True)
    (21): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (22): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (23): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (24): ReLU(inplace=True)
    (25): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (26): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (27): ReLU(inplace=True)
    (28): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (avgpool): AdaptiveAvgPool2d(output_size=7)
  (classifier): Sequential(
    (0): Linear(in_features=25088, out_features=4096, bias=True)
    (1): ReLU(inplace=True)
    (2): Dropout(p=0.5, inplace=False)
    (3): Linear(in_features=4096, out_features=2, bias=True)
  )
)
```

완전 연결층 & 출력층

#6.1.3 VGGNet

VGG 11 구성 코드:

사전 훈련된 모델 불러와서

사용

```
import torchvision.models as models
pretrained_model = models.vgg11(pretrained=True)
print(pretrained_model)
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The p
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Argum
warnings.warn(msg)
```

```
VGG(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): ReLU(inplace=True)
    (10): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (11): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (12): ReLU(inplace=True)
    (13): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (14): ReLU(inplace=True)
    (15): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (16): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (17): ReLU(inplace=True)
    (18): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (19): ReLU(inplace=True)
    (20): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(7, 7))
  (classifier): Sequential(
    (0): Linear(in_features=25088, out_features=4096, bias=True)
    (1): ReLU(inplace=True)
    (2): Dropout(p=0.5, inplace=False)
    (3): Linear(in_features=4096, out_features=4096, bias=True)
    (4): ReLU(inplace=True)
    (5): Dropout(p=0.5, inplace=False)
    (6): Linear(in_features=4096, out_features=1000, bias=True)
  )
)
```


#6.1.3 VGGNet

데이터 전처리

```
train_transforms = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomRotation(5),
    transforms.RandomHorizontalFlip(0.5),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
```

```
test_transforms = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
```

```
train_path = '/content/drive/MyDrive/Euron/catanddog/train'
test_path = '/content/drive/MyDrive/Euron/catanddog/test'
```

```
train_dataset = torchvision.datasets.ImageFolder(
    train_path,
    transform=train_transforms
)
```

(module) datasets

```
test_dataset = torchvision.datasets.ImageFolder(
    test_path,
    transform=test_transforms
)
```

```
print(f"Length of train_dataset: {len(train_dataset)}")
print(f"Length of test_dataset: {len(test_dataset)}")
```

```
Length of train_dataset: 529
Length of test_dataset: 12
```

훈련용 데이터 529개
테스트용 데이터 12개

transforms.Resize(n, n) : nxn크기로 이미지 조정
transforms.RandomRotation(n) : n도 이하로 이미지 회전
transforms.RandomHorizontalFlip(n) : n확률로 이미지 수평으로 뒤집음
transforms.ToTensor() : NumPy배열을 PyTorch 텐서로 변환
transforms.Normalize(mean = [], std=[]) : 텐서의 픽셀 값 정규화,
평균과 표준편차 값은 ImageNet데이터셋에서 미리 계산된 값임

#6.1.3 VGGNet

데이터 전처리

```
VALID_RATIO = 0.9
n_train_examples = int(len(train_dataset) * VALID_RATIO)
n_valid_examples = len(train_dataset) - n_train_examples

train_data, valid_data = data.random_split(train_dataset, [n_train_examples, n_valid_examples])

valid_data = copy.deepcopy(valid_data)
valid_data.dataset.transform = test_transforms
```

```
print(f'Number of training examples: {len(train_data )}')
print(f'Number of validation examples: {len(valid_data)}')
print(f'Number of testing examples: {len(test_dataset)}')
```

```
Number of training examples: 476
Number of validation examples: 53
Number of testing examples: 12
```

훈련, 검증, 테스트 데이터셋 수 확인

```
BATCH_SIZE = 128
train_iterator = data.DataLoader(train_data, shuffle=True, batch_size=BATCH_SIZE)
valid_iterator = data.DataLoader(valid_data, batch_size=BATCH_SIZE)
test_iterator = data.DataLoader(test_dataset, batch_size=BATCH_SIZE)
```

데이터 분할:

- 전체 훈련 데이터 중 90%를 훈련 데이터셋으로 사용
- 전체 훈련 데이터 중 10%를 검증 데이터셋으로 사용
- random_split으로 데이터 분리

DataLoader: 데이터셋의 데이터를 배치 크기 만큼 나눠서 메모리로 가져옴

#6.1.3 VGGNet

모델 학습 준비

예측이 정답과 일치하는 경우 그 개수의 합을 correct 변수에 저장

#모델 학습에 사용할 옵티마이저와 손실함수 정의

```
optimizer = optim.Adam(model.parameters(), lr=1e-7)
criterion = nn.CrossEntropyLoss()
```

```
model = model.to(device)
criterion = criterion.to(device)
```

#모델 정확도 측정 함수 정의

```
def calculate_accuracy(y_pred, y):
    top_pred = y_pred.argmax(1, keepdim=True)
    correct = top_pred.eq(y.view_as(top_pred)).sum()
    acc = correct.float() / y.shape[0]
    return acc
```

#훈련 데이터셋을 이용한 모델 학습 함수 정의

```
def train(model, iterator, optimizer, criterion, device):
    epoch_loss=0
    epoch_acc=0

    model.train()
    for (x, y) in iterator:
        x = x.to(device)
        y = y.to(device)

        optimizer.zero_grad()
        y_pred, _ = model(x)
        loss = criterion(y_pred, y)
        acc = calculate_accuracy(y_pred, y)

        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()
        epoch_acc += acc.item()
    return epoch_loss/len(iterator), epoch_acc/len(iterator)
```

#검증 및 테스트 데이터셋을 이용한 모델 성능을 측정하는 함수 정의

```
def evaluate(model, iterator, criterion, device):
    epoch_loss = 0
    epoch_acc = 0

    model.eval()
    with torch.no_grad():
        for (x, y) in iterator:
            x = x.to(device)
            y = y.to(device)
            y_pred, _ = model(x)
            loss = criterion(y_pred, y)
            acc = calculate_accuracy(y_pred, y)
            epoch_loss += loss.item()
            epoch_acc += acc.item()

    return epoch_loss/len(iterator), epoch_acc/len(iterator)
```

#모델의 학습 시간(시작, 종료)을 측정하기 위한 함수 정의

```
def epoch_time(start_time, end_time):
    elapsed_time = end_time - start_time
    elapsed_mins = int(elapsed_time/60)
    elapsed_secs = int(elapsed_time - (elapsed_mins * 60))
    return elapsed_mins, elapsed_secs
```

#6.1.3 VGGNet

모델 학습

```
EPOCHS = 5
best_valid_loss = float('inf')
for epoch in range(EPOCHS):
    start_time = time.monotonic()
    train_loss, train_acc = train(model, train_iterator, optimizer, criterion, device)
    valid_loss, valid_acc = evaluate(model, valid_iterator, criterion, device)

    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model.state_dict(), '../chap06/data/VGG-model.pt') #valid loss가 가장 작은 값을 구하고
                                                                    #그 상태의 모델을 VGG-model.pt 이름으로 저장

end_time = time.monotonic()
epoch_mins, epoch_secs = epoch_time(start_time, end_time)

print(f'Epoch: {epoch+1:02} | Epoch Time: {epoch_mins}m {epoch_secs}s')
print(f'\tTrain Loss: {train_loss:.3f} | Train Acc: {train_acc*100:.2f}%')
print(f'\t Valid Loss: {valid_loss:.3f} | Valid Acc: {valid_acc*100:.2f}%')
```

```
Epoch: 01 | Epoch Time: 8m 27s
      Train Loss: 0.691 | Train Acc: 54.48%
      Valid. Loss: 0.693 | Valid. Acc: 52.83%
Epoch: 02 | Epoch Time: 7m 40s
      Train Loss: 0.690 | Train Acc: 55.38%
      Valid. Loss: 0.693 | Valid. Acc: 52.83%
Epoch: 03 | Epoch Time: 7m 34s
      Train Loss: 0.699 | Train Acc: 50.73%
      Valid. Loss: 0.693 | Valid. Acc: 60.38%
Epoch: 04 | Epoch Time: 7m 33s
      Train Loss: 0.689 | Train Acc: 53.29%
      Valid. Loss: 0.693 | Valid. Acc: 47.17%
Epoch: 05 | Epoch Time: 7m 28s
      Train Loss: 0.700 | Train Acc: 51.52%
      Valid. Loss: 0.693 | Valid. Acc: 47.17%
```

#테스트 데이터셋에 대한 VGG-model.pt의 성능 측정

```
model.load_state_dict(torch.load('../chap06/data/VGG-model.pt'))
test_loss, test_acc = evaluate(model, test_iterator, criterion, device)
print(f'Test Loss: {test_loss:.3f} | Test acc: {test_acc*100:.2f}%')
```

Test Loss: 0.693 | Test Acc: 50.00%

예측

torch.cat

```
with torch.no_grad():
    for (x, y) in iterator:
        x = x.to(device)
        y_pred, _ = model(x)
        y_prob = F.softmax(y_pred, dim=-1)
        top_pred = y_prob.argmax(1, keepdim=True)
        images.append(x.cpu())
        labels.append(y.cpu())
        probs.append(y_prob.cpu())

images = torch.cat(images, dim=0)
labels = torch.cat(labels, dim=0)
probs = torch.cat(probs, dim=0)
return images, labels, probs
```

```
x = torch.Tensor([[1, 2, 3], [2, 3, 4]])
y = torch.Tensor([[4, 5, 6], [5, 6, 7]])
```

```
tensor([[1., 2., 3.],
        [2., 3., 4.]])
```

print(torch.cat([x], dim=0)) ----- 행을 기준(dim=0)으로 x를 이어 붙입니다.

```
print(torch.cat([x, y], dim=0)) ----- 행을 기준(dim=0)으로 x와 y를 이어 붙이면 (4×3) 텐서가 됩니다
print('-----')
```

print(torch.cat([x, y], dim=1)) ----- 열을 기준(dim=1)으로 x와 y를 이어 붙이면 (2×6) 텐서가 됩니다.

```
tensor([[1., 2., 3.],
        [2., 3., 4.],
        [4., 5., 6.],
        [5., 6., 7.]])
```

```
tensor([[1., 2., 3., 4., 5., 6.],
        [2., 3., 4., 5., 6., 7.]])
```

#6.1.3 VGGNet

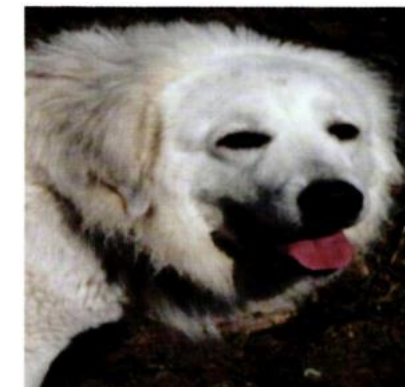
예측

```
#get_predictions함수의 반환값 각각 저장하여 정확하게 예측한 이미지 추출
images, labels, probs = get_prediction(model, test_iterator)
pred_labels = torch.argmax(probs, 1)
corrects = torch.eq(labels, pred_labels)
correct_examples=[]

for image, label, prob, correct in zip(images, labels, probs, corrects):
    if correct:
        correct_examples.append((image, label, prob))

correct_exmples.sort(revers=True, key=lambda x:torch.max(x[2], dim=0).values)

def normalize_image(image):
    image_min = image.min()
    image_max = image.max()
    image.clamp_(min=image_min, max=image_max)
    image.add_(-image_min).div_(image_max - image_min + 1e-5)
    return image
```



#6.1.3 VGGNet

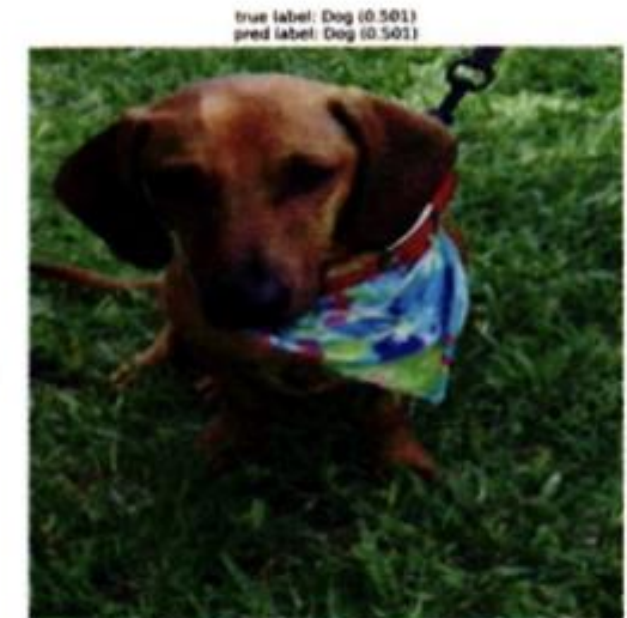
예측

```
def plot_most_correct(correct, classes, n_images, normalize=True):
    rows = int(np.sqrt(n_images))
    cols = int(np.sqrt(n_images))
    fig = plt.figure(figsize=(25, 20))
    for i in range(rows*cols):
        ax = fig.add_subplot(rows, cols, i+1)
        image, label, prob = correct[i]
        image = image.permute(1, 2, 0)
        true_prob, true_label = torch.max(prob, dim=0)
        true_class = classes[true_label]
        correct_class = classes[correct_label]

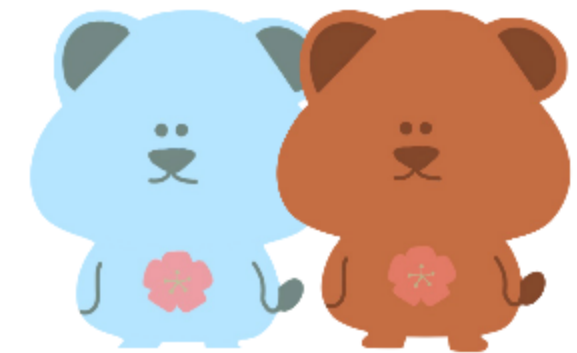
        if normalize:
            image = normalize_image(image)
        ax.imshow(image.cpu().numpy())
        ax.set_title(f'true label: {true_class} ({true_prob:.3f})\n' \
                    f'pred label: {correct_class} ({correct_prob:.3f})')

        ax.axis('off')

    fig.subplots_adjust(haspace=0.4)
```



6.1.4 GoogLeNet



#6.1.4 GoogLeNet

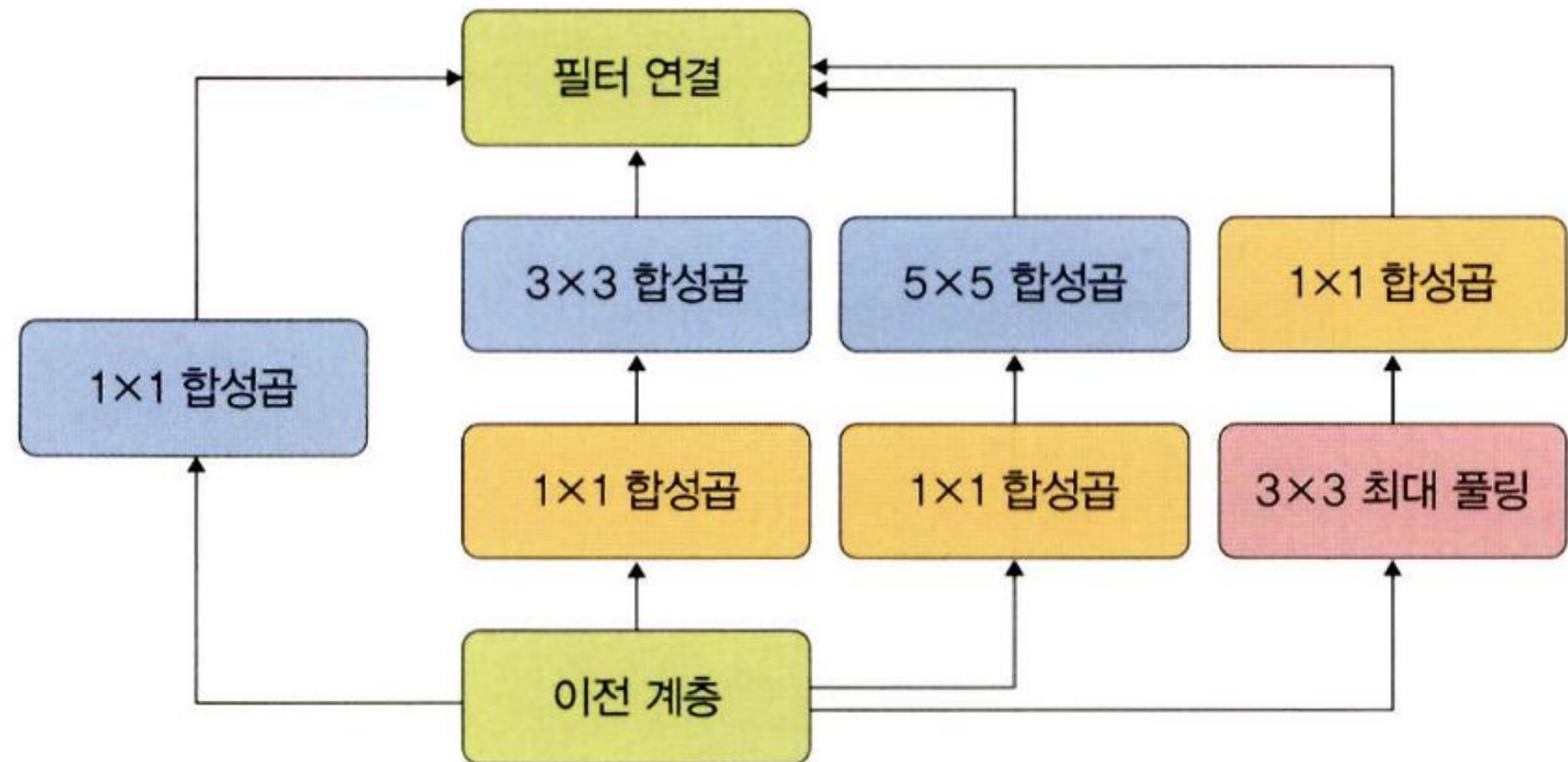
심층 신경망의 아키텍처에서:
계층이 깊고 뉴런이 많으면 높은 인식률

- 과적합
- 기울기 소멸 문제
- 시간 지연/ 연산 속도

GoogLeNet: 주어진 하드웨어의 **자원을 최대한 효율적으로** 이용하면서 **학습 능력은 극대화**할 수 있는 깊고 넓은 신경망

인셉션 모듈

하나의 층에서 여러 크기의 합성곱을 동시에 수행
-> 다양한 특징을 한 번에 추출 할 수 있음



#6.1.4 GoogLeNet

인셉션 모듈의 4가지 연산:

1x1합성곱

입력 채널을 줄이는 차원 축소 역할
계산량 감소
비선형성 추가

1x1합성곱 + 3x3합성곱

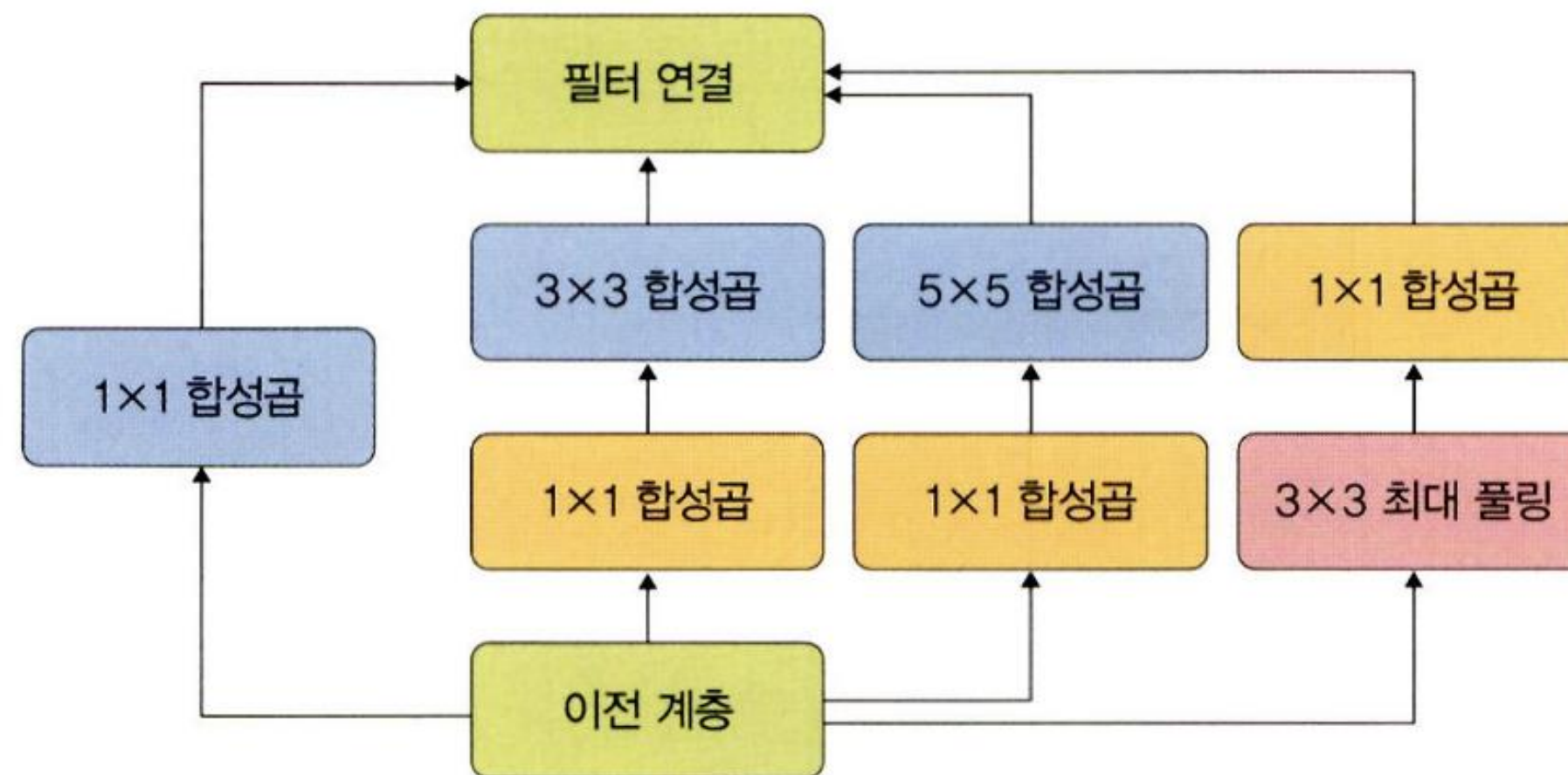
1x1로 채널 수 축소 -> 3x3으로 특징 추출.
(중간 크기 패턴 학습)

1x1합성곱 + 5x5합성곱

1x1로 채널 수 축소 -> 5x5으로 특징 추출.
(넓은 영역의 패턴 학습)

3x3 최대 풀링 + 1x1합성곱

3x3 max pooling: 중요 특징만 유지
1x1합성곱: 채널 수 조정 및 정보 보완
(공간 정보 유지하면서 강한 특징 강조)



네가지 연산을 동시에 수행 + 결과를 채널 방향으로 결합 -> 한번에 다양한 스케일의 특징 추출 가능

THANK YOU

